

Universidad Tecnológica Del Valle de Toluca

Carrera: Ingeniería en Desarrollo y Gestión de Software

Reporte de Evidencias de Práctica

Nombre de la práctica:

Terraform

Nombre del estudiante:

Jesus Felipe Aviles

Materia:

Aplicaciones Web Progresivas

Profesor(a):

MTI. Jorge Almeida Montiel

Fecha: 25 de septiembre de 2026

Contenido

Introducción.....	3
Objetivo de la práctica	3
Herramientas y materiales utilizados	3
Desarrollo de la práctica	4
Evidencias fotográficas / capturas de pantalla	5
Resultados obtenidos	7
Análisis de resultados	8
Conclusiones	8

Introducción

En este documento se presenta el reporte de evidencias de la práctica realizada utilizando Terraform, con el objetivo de documentar de manera formal el proceso de configuración, ejecución y validación de un proyecto de Infraestructura como Código (Infrastructure as Code, IaC). El reporte incluye la descripción del procedimiento realizado, las herramientas utilizadas, las evidencias obtenidas y los resultados alcanzados durante el desarrollo de la práctica.

Objetivo de la práctica

El objetivo principal de esta práctica es conocer los fundamentos de Terraform y aplicar el concepto de Infraestructura como Código mediante la creación y configuración de un proyecto básico. Asimismo, se busca comprender la estructura de los archivos de configuración de Terraform, conocer el funcionamiento de los proveedores y recursos, inicializar correctamente un proyecto y verificar que la configuración establecida pueda ser ejecutada correctamente.

Herramientas y materiales utilizados

Durante la práctica se utilizaron los siguientes elementos:

- Terraform.
- Visual Studio Code.
- Sistema operativo Windows.
- PowerShell o terminal de comandos.
- Lenguaje de configuración HCL (HashiCorp Configuration Language).
- Proveedor hashicorp/random.
- Conexión a Internet para la descarga de dependencias necesarias.
- Equipo de cómputo utilizado para el desarrollo de la práctica.

Desarrollo de la práctica

La práctica se desarrolló mediante la creación de un proyecto básico utilizando Terraform. Inicialmente se preparó el entorno de trabajo y se verificó que Terraform estuviera instalado y disponible desde la terminal.

Posteriormente se creó un archivo de configuración denominado `main.tf`, en el cual se establecieron los requisitos de Terraform, el proveedor que sería utilizado y el recurso que se deseaba crear.

Para esta primera práctica se utilizó el proveedor `hashicorp/random`, debido a que permite generar valores aleatorios que pueden ser administrados como recursos mediante Terraform. Dentro de la configuración se definió un recurso `random_string`, estableciendo una longitud de 16 caracteres.

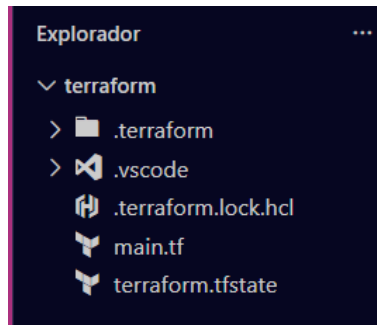
Después de crear la configuración, se realizó la inicialización del proyecto para descargar y configurar el proveedor requerido. Finalmente, se validó la configuración y se ejecutó Terraform para comprobar que el recurso definido pudiera ser creado correctamente.

Procedimiento realizado

1. Se preparó el entorno de trabajo para utilizar Terraform.
2. Se verificó que Terraform estuviera instalado correctamente mediante la terminal.
3. Se creó una carpeta para almacenar los archivos correspondientes al proyecto.
4. Se creó el archivo `main.tf`.
5. Se configuró el bloque `terraform` para establecer los requisitos de la herramienta.
6. Se agregó el proveedor `hashicorp/random`.
7. Se definió el recurso `random_string`.
8. Se estableció una longitud de 16 caracteres para el valor generado.
9. Se ejecutó el comando `terraform init` para inicializar el proyecto y descargar el proveedor necesario.
10. Se ejecutó `terraform validate` para comprobar que la configuración fuera válida.
11. Se ejecutó Terraform para crear el recurso definido.
12. Se verificó que la ejecución terminara correctamente y que el recurso fuera administrado por Terraform.
13. Se revisaron los archivos generados durante la inicialización y ejecución del proyecto.

Evidencias fotográficas / capturas de pantalla

En esta sección se insertan las capturas de pantalla relevantes, como, por ejemplo:



```
● PS C:\Users\jesus\Documents\PWA\terraform> terraform init
Initializing the backend...
```

Initializing provider plugins...

- Finding hashicorp/random versions matching "~> 3.0"...
- Installing hashicorp/random v3.9.1...
- Installed hashicorp/random v3.9.1 (signed by HashiCorp)

Terraform has created a lock file `.terraform.lock.hcl` to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
● PS C:\Users\jesus\Documents\PWA\terraform> terraform validate
Success! The configuration is valid.
```

```
● PS C:\Users\jesus\Documents\PWA\terraform> terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

Terraform will perform the following actions:

```
# random_string.suffix will be created
+ resource "random_string" "suffix" {
  + id          = (known after apply)
  + length      = 16
  + lower       = true
  + min_lower   = 0
  + min_numeric = 0
  + min_special = 0
  + min_upper   = 0
  + number      = true
  + numeric     = true
  + result      = (known after apply)
  + special     = true
  + upper       = true
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the `-out` option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

```
PS C:\Users\jesus\Documents\PWA\terraform> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# random_string.suffix will be created
+ resource "random_string" "suffix" {
  + id          = (known after apply)
  + length      = 16
  + lower       = true
  + min_lower   = 0
  + min_numeric = 0
  + min_special = 0
  + min_upper   = 0
  + number      = true
  + numeric     = true
  + result      = (known after apply)
  + special     = true
  + upper       = true
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

random_string.suffix: Creating...
random_string.suffix: Creation complete after 0s [id=XvL(A&*IjX_%\NJ<?)]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Resultados obtenidos

Durante la práctica se obtuvieron los siguientes resultados:

- Se creó correctamente un proyecto básico utilizando Terraform.
- Se configuró el proveedor hashicorp/random.
- Se definió correctamente un recurso de tipo random_string.
- Se estableció una longitud de 16 caracteres para el valor generado.
- Se inicializó correctamente el proyecto mediante terraform init.
- Se verificó la configuración mediante terraform validate.
- Se ejecutó correctamente la creación del recurso mediante Terraform.
- Se comprobó el funcionamiento básico del flujo de trabajo de Terraform.

Se evidenció que la actividad fue realizada correctamente y que se logró comprender el proceso básico de definición, inicialización, validación y ejecución de una infraestructura mediante código.

Análisis de resultados

Los resultados obtenidos permiten confirmar que Terraform puede utilizarse para definir y administrar recursos mediante archivos de configuración, evitando la necesidad de realizar todas las acciones manualmente.

Durante la práctica se observó que el archivo `main.tf` funciona como el elemento principal de configuración, ya que en él se especifican los requisitos de Terraform, los proveedores necesarios y los recursos que serán administrados.

La utilización del proveedor `hashicorp/random` permitió trabajar con un recurso sencillo y observar de manera práctica el funcionamiento de Terraform. El recurso `random_string` generó un valor aleatorio de 16 caracteres de acuerdo con la configuración establecida.

La práctica también permitió comprender la importancia de inicializar el proyecto antes de ejecutar los recursos, ya que mediante `terraform init` Terraform prepara el entorno y obtiene las dependencias necesarias.

Finalmente, mediante `terraform validate` y `terraform apply` fue posible comprobar que la configuración era válida y que el recurso podía ser creado correctamente.

Conclusiones

La práctica fue desarrollada de manera satisfactoria, cumpliendo con los objetivos planteados. Mediante la creación de un proyecto básico fue posible conocer los fundamentos de Terraform y comprender el concepto de Infraestructura como Código.

La configuración del proveedor `hashicorp/random` y del recurso `random_string` permitió trabajar con una implementación sencilla para comprender cómo Terraform define y administra recursos mediante archivos de configuración.

Asimismo, la ejecución de los comandos `terraform init`, `terraform validate` y `terraform apply` permitió conocer las principales etapas del flujo de trabajo de Terraform, desde la inicialización del proyecto hasta la creación y validación de los recursos.

La experiencia adquirida durante esta práctica proporciona una base para continuar trabajando con Terraform en proyectos más complejos, donde se pueden administrar recursos de infraestructura, servicios en la nube, variables, módulos y diferentes proveedores.